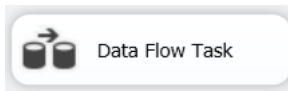


# Laboratory of Data Science

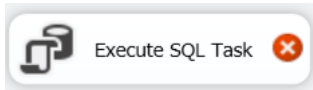
## cheatsheet

# 1 SSIS Toolbox

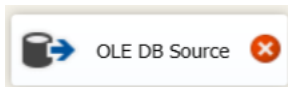
**Data Flow Task** Used to move data between sources and destinations. Allows to perform data cleaning and transformations.



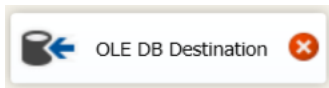
**Execute SQL Task** Used to execute SQL queries. Can be combined with Data Flow Task to perform ETL operations. Queries can be specified in the editor, under the SQLStatement property.



**OLE DB Source** Connects and reads data from and OLE DB database. This component allows the user to choose which table and which columns to read.



**OLE DB Destination** Connects and writes data to an OLE DB database. This component allows the user to choose which table to write to, as well as the mapping between the input columns and the destination columns.



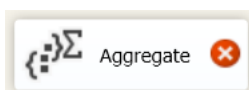
**Flat File Source** Reads data from flat files (e.g., csv, txt). The user can specify the file path, and which columns/rows to read.



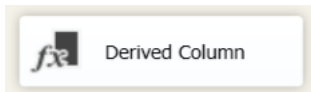
**Flat File Destination** Writes data to flat files (e.g., csv, txt). The user can specify the file path, and which columns/rows to write.



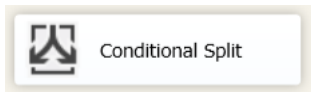
**Aggregate** Groups input rows based on the specified columns and aggregates data using a set of available functions (e.g., sum, average, count), depending on the data type.



**Derived Column** Creates new columns or replaces existing ones with calculated expressions. The editor provides a set of functions and operators to build expressions.



**Conditional Split** Splits the data flow into multiple paths based on specified conditions. It behaves like a switch/case statement, and allows a default output for rows that do not match any condition.



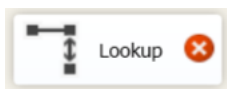
**Multicast** Multiplies the input data and sends it to different outputs.



**Sort** Sorts the input data based on the specified columns and order (ascending/descending). Also allows to remove duplicate rows.



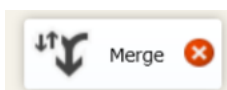
**Lookup** Performs a lookup of input rows against a remote source. It is normally used to add columns to the already available ones. Since it's a lookup, even if a row matches multiples in the source, only the first match is returned. If a row has no match, the editor allows to choose how to handle it (ignore, fail, redirect).



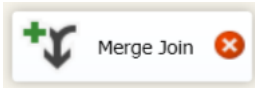
**Union All** Combines rows from multiple inputs in a single output. It is the equivalent of a SQL UNION ALL operation.



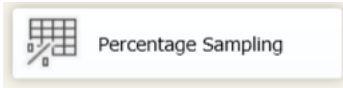
**Merge** Similar to Union All, but requires input rows to be sorted, and produces a sorted output.



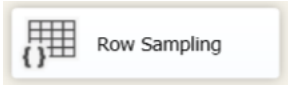
**Merge Join** Implements a join operation (inner, left, full outer), producing only rows that satisfy the join condition. Requires input rows to be sorted.



**Percentage Sampling** Randomly selects the specified percentage of rows from the input data.



**Row Sampling** Randomly selects a specified number of rows from the input data.



## 2 MDX Functions

### Numeric functions

- `Aggregate(set_exp, [, numeric_exp])`: applies an aggregation function to the cells returned by the `set_expression`. If the `numeric_expression` is not specified, the default aggregation function of the measure is used. If a `numeric_expression` is specified, the aggregation is first applied to each cell, and then summed across them.
- `Sum(set_exp, [, numeric_exp])`: returns the sum of a `numeric_expression` evaluated for each cell in a set. If the `numeric_expression` is not specified, the plain value of each cell is used. Analogous functions exist for other aggregations: `Avg`, `Min`, `Max`.
- `Count(set_exp, [, (excludeempty | includeempty)])`: returns the number of cells in a set. It can be set to include or exclude empty cells from the result.
- `DistinctCount(set_exp)`: returns the number of distinct, nonempty tuples in a set.
- `Rank(tuple_exp, set_exp, [, numeric_exp])`: returns the rank of a tuple within a set; if a `numeric_expression` is specified, the rank is determined by evaluating the expression against the tuple, otherwise the rank is the ordinal position of the tuple in the set.

### 2.1 Hierarchy navigation

- `hierarchy_exp.CurrentMember`: returns the current member in the specified hierarchy during iteration.
- `hierarchy_exp.All`: returns the “All” member of the specified hierarchy.
- `member_exp.Parent`: returns the parent of the current member in the hierarchy.
- `Ancestor(member_exp, (level_exp | distance))`: returns the ancestor of a member at a specified level or distance from the member.
- `member_exp.Children`: returns the set of children of the member. If the member has no children, an empty set is returned.
- `Descendants(member_exp, [, (level_exp | distance) [, desc_flag]])`: returns the set of descendants of a hierarchy member at a specified level or distance, optionally including or excluding descendants in intermediate levels.

- `member_exp.Siblings`: returns the siblings of a specified member, including the member itself.
- `Cousin(member_exp, ancestor_exp)`: returns the child member that shares the same relative position as the specified member under a different ancestor.
- `member_exp.Lag(idx)`: returns the member that is *idx* positions before the specified member within the same level of the hierarchy.
- `member_exp.Lead(idx)`: returns the member that is *idx* positions after the specified member within the same level of the hierarchy.
- `LinkMember(member_exp, hierarchy_exp)`: returns the member in the hierarchy that is equivalent to the specified member.
- `hierarchy_exp.UnknownMember`: returns the “Unknown” member in a hierarchy level, if it exists.

## 2.2 Set functions

- `Crossjoin(set_exp1, set_exp2 [, ...n])`: returns the cross product of the input sets. An alternate syntax is: `set_exp1 * set_exp2 * ... * set_expn`. If the sets are composed of tuples from different hierarchies in the same dimension, the output will only contain tuples that actually exist.
- `Union(set_exp1, set_exp2 [, ...n])[, ALL]`: returns a set that contains the union of the input sets. Optionally, duplicates can be preserved using the ALL flag. An alternate syntax is: `set_exp1 + set_exp2 + ... + set_expn`.
- `Intersect(set_exp1, set_exp2, [, ALL])`: returns the intersection of two input sets, optionally preserving duplicates.
- `Filter(set_exp, logical_exp)`: returns the tuples in the input set that satisfy a search condition.
- `Exists(set_exp1, set_exp2)`: returns the set of tuples of the first set that exist in the second.
- `Except(set_exp1, set_exp2, [, ALL])`: removes the tuples in the first set that also exist in the second, optionally retaining duplicates.
- `hierarchy_exp.members` or `level_exp.members`: returns the set of members in a dimension, level, or hierarchy.
- `Distinct(set_exp)`: removes duplicates from the input set and returns the resulting set.
- `NONEMPTY(set_exp1 [, set_exp2])`: returns the set of tuples that are not empty in a specified set, optionally based on the cross product with a second set.
- `Order(set_exp, numeric_exp [, ASC | DESC | BASC | BDESC])`: orders the tuples of the input set based on the evaluation of a numeric expression over each tuple. THE B in BASC and BDESC stands for “break hierarchy”, meaning that ordering does not consider the hierarchy structure of the members.
- `Head(set_exp, [, count])`: returns the first specified number (*count*) of tuples in the specified set, retaining duplicates.

- **Tail(set\_exp, [, count]):** returns the last specified number (**count**) of tuples in the specified set, retaining duplicates.
- **TopCount(set\_exp, count [, numeric\_exp]):** sorts a set in descending order and returns the top **count** elements with the highest values. If a numeric expression is specified, ordering is based on the evaluation of the expression over each tuple. If no expression is specified, the members are considered in their natural order, without any sorting.
- **TopSum(set\_exp, value, numeric\_exp):** sorts a set and returns the topmost elements whose cumulative total is at least **value**. Ordering is based on the evaluation of the numeric expression over each tuple. The function then takes the smallest subset of the ordered set whose cumulative total is at least **value**.
- **Generate(set\_exp1, set\_exp2, [, ALL]):** applies the tuples in the second set to each member in the first set, then joins the resulting sets by union. If the **ALL** flag is specified, duplicates are preserved. This function is often used to evaluate complex set expression over a set of members (e.g., the top 10 customers by sales for each country).

## 2.3 Time navigation

- **ParallelPeriod([level\_exp, [, index, [, member\_exp]]]):** returns a member from a prior period in the same relative position as the specified member. It is similar to **Cousin**, but is specifically designed for time series: **ParallelPeriod** finds the ancestor of the specified member at the given level, then finds the ancestor's sibling with the specified lag (**index**), and returns the parallel period of the member along the descendants of the sibling. If no level or member expression are specified, the default member is the current member of the first hierarchy on the first dimension with a type of **Time** in the measure group.
- **PeriodsToDate([level\_exp,] member\_exp):** returns the set of siblings with the same level as a given member, starting from the first and ending at the given member, constrained by the given level. If only the level is specified, the member is automatically assumed to be the current member in the iteration. If neither level nor member are specified, the level is the parent level of the current member of the first hierarchy on the first dimension with a type of **Time** in the measure group.

## Other

- **IIf(logical\_exp, exp1, exp2):** evaluates a logical expression; if it is true, returns the result of **exp1**, otherwise returns the result of **exp2**.
- **IsEmpty(value\_exp):** returns whether the evaluated expression is the empty cell.
- **format\_string:** a cell property used to control the display format of numeric values in the result set. It allows to both use predefined formats (e.g., "Currency", "Percent") and custom formats.